

Resource-Aware Cooperative World Models at the Edge

Anonymous Authors

Abstract—Edge-hosted cooperative world models that fuse intermediate features from multiple connected autonomous vehicles (CAVs) enable perception beyond line-of-sight. However, as the number of participating agents grows, the full edge pipeline (spatial-attention fusion, multi-object tracking, transformer trajectory prediction) encounters a steep deadline cliff: on a commodity GPU the static pipeline exceeds a 100 ms planner deadline at roughly $N = 10$ agents, and latency grows super-linearly thereafter. We present a deadline-aware adaptive controller with two decision variables. An input contribution filter selects the K most informative agents per tick using conflict-zone scoring. A risk-budgeted scheduler allocates the residual compute budget to the subset of tracked objects whose cached predictions have diverged, prioritized by a continuous risk score combining speed, proximity, occlusion, and conflict geometry. Temporal amortization gates fresh neural-network inference to divergent objects only, reusing cached predictions for stable tracks. Evaluated offline on real post-backbone features across $N \in \{4, 8, 16, 24, 32, 48, 64\}$ agents, the adaptive controller maintains 100% deadline compliance while the static baseline fails at $N \geq 16$. Risk-stratified quality metrics confirm that prediction accuracy for safety-critical objects is preserved under adaptation.

I. INTRODUCTION

Connected autonomous vehicles (CAVs) benefit from cooperative perception: sharing sensor data or intermediate feature representations across a vehicular network transcends physical limitations such as occlusion, limited sensor range, and adverse weather. Recent cooperative perception-prediction stacks such as CMP [1], [2] demonstrate that aggregating multi-agent data yields measurably better tracking and forecasting than single-agent systems.

Deploying these stacks at dense scale exposes two infrastructure bottlenecks. First, direct vehicle-to-vehicle (V2V) exchange of intermediate feature maps is physically infeasible under dense participation: our ns-3 5G-LENA NR V2X sidelink simulations show a Packet Reception Ratio (PRR) of zero for 200 KB payloads at every tested N , because a single intermediate feature map exceeds the sidelink transport block capacity. Second, while centralizing on a Multi-Access Edge Computing (MEC) server resolves the bandwidth constraint, it introduces a deadline cliff: the full edge pipeline (spatial-attention fusion, multi-object tracking, trajectory prediction) scales super-linearly with the number of agents, crossing a 100 ms planner deadline at $N \approx 10$ on commodity hardware and reaching 184 ms at $N = 64$.

The edge server possesses three assets unavailable to individual vehicles: (i) persistent Kalman-filtered track state across ticks, enabling divergence-gated prediction reuse; (ii) a global scene graph containing all ego poses, planned paths, and fields of view; and (iii) a single centralized compute budget governed by one deadline. These assets make the edge the natural locus for runtime adaptation.

We present a deadline-aware adaptive controller with two decision variables: K , the number of agents retained after an input contribution filter, and $x_i \in \{0, 1\}$, a per-object binary indicating whether tracked object i receives fresh transformer prediction or reuses a cached trajectory. The controller implements three mechanisms:

- 1) **Input contribution filtering.** Score each agent by detection confidence within the ego-vehicle conflict zone; retain the top- K agents that maximize estimated downstream utility.
- 2) **Temporal amortization.** Gate fresh prediction inference on per-object divergence between the observed track state and the cached prediction, using velocity-normalized positional and heading thresholds.
- 3) **Risk-budgeted scheduling.** Allocate the residual prediction budget (after fusion and tracking) to divergent objects ranked by a continuous risk score integrating speed, proximity to ego, occlusion status, and conflict geometry.

Rate adaptation (reducing the global tick frequency) is retained as a fallback triggered only when no feasible K keeps the estimated pipeline cost within the deadline.

We evaluate the controller offline on real post-backbone features from the OPV2V dataset, using a WorldFusion (Where2Comm attention) fusion model, AB3DMOT tracking, and MTR transformer prediction. Across $N \in \{4, \dots, 64\}$, the adaptive controller maintains 100% deadline compliance at a 100 ms budget while the static baseline fails at $N \geq 16$. A fixed-ground-truth evaluation with explicit miss-rate penalties and risk stratification confirms that prediction quality for high-risk objects is preserved under adaptation.

Our contributions:

- 1) **Dense-scale characterization.** Per-stage latency measurements for a WorldFusion + AB3DMOT + MTR edge pipeline, identifying the deadline cliff and its per-component drivers.
- 2) **Adaptive controller.** A two-variable greedy controller (input filter K , per-object prediction x_i) that maintains deadline compliance through $N = 64$

while preserving safety-critical prediction quality.

- 3) **Risk-stratified evaluation methodology.** A fixed-GT evaluation protocol that penalizes missing predictions and separates quality reporting by object risk level.

II. PROBLEM SETTING

A. V2V Intermediate Fusion is Infeasible at Dense Scale

Intermediate fusion requires each vehicle to transmit a compressed Bird's Eye View (BEV) feature map to its peers. Even after post-backbone compression ($256\times$ ratio, yielding ~ 17 KB per vehicle), the aggregate sidelink demand at 10Hz with N agents scales as $O(N^2)$ because every agent broadcasts to every other. At $N = 40$ and 17KB payloads, total sidelink demand reaches ~ 54 Mbps, exceeding C-V2X PC5 capacity (~ 50 Mbps) and triggering MAC-layer collapse.

We validate with ns-3 5G-LENA NR V2X Mode 2 simulations. For uncompressed 200 KB intermediate features, the Packet Reception Ratio (PRR) is exactly zero at every N : a single feature map exceeds the sidelink transport block size. Even with 17 KB compressed payloads, PRR degrades from 0.30 at $N = 2$ to 0.006 at $N = 64$. The finding is categorical: V2V intermediate fusion at dense scale is physically unviable.

B. Edge Placement Resolves Bandwidth, Introduces Deadline Cliff

Centralizing fusion on an edge server connected via 5G NR Uu uplinks eliminates the $O(N^2)$ broadcast problem: each vehicle uploads once, and the edge returns a lightweight object list (~ 5 KB). Uu scheduled access provides contention-free bandwidth sharing that scales gracefully with N .

The edge pipeline, however, operates under a planner deadline D (typically 100 ms at 10Hz). We define:

- **Generation time** (t_{gen}): sensor sampling instant.
- **Publish time** (t_{pub}): instant the edge broadcasts the world model update.
- **Use time** (t_{use}): instant the ego vehicle's planner ingests the state.

The Age-of-Information at use is $\Delta_{\text{use}} = t_{\text{use}} - t_{\text{gen}}$. A **deadline miss** occurs when $\Delta_{\text{use}} > D$.

When the total edge compute time $T_{\text{compute}}(N)$ exceeds D , cross-stage queueing drives Δ_{use} unboundedly upward. On our hardware (§V), the static pipeline crosses 100 ms at $N \approx 10$.

C. Planner-Aware Utility

We define a risk-weighted, deadline-qualified utility to drive the adaptive controller (§IV):

$$U(K, \mathbf{x}) = \sum_{i=1}^M r_i \cdot q_i(x_i, a_i) \quad (1)$$

where r_i is the continuous risk score of tracked object i (§IV-C), q_i is the prediction quality (1.0 for fresh MTR inference, decaying with cache age a_i , floor $q_{\text{lin}} \approx 0.3$ for linear extrapolation), and utility is implicitly zero

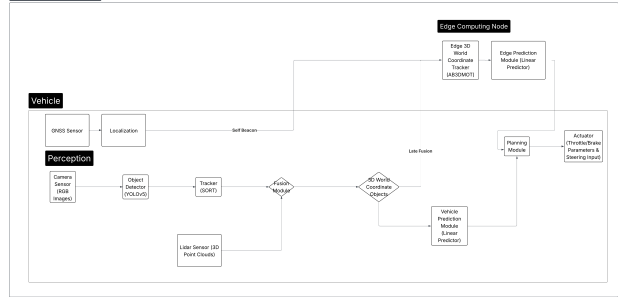


Fig. 1: Edge cooperative world-model pipeline. Each CAV encodes LiDAR into a compressed post-backbone BEV feature (17 KB) and uploads via 5G Uu. The edge runs WorldFusion (Where2Comm) attention fusion, AB3DMOT tracking, and MTR trajectory prediction. The adaptive controller selects which agents and objects to process within the 100 ms planner deadline.

when $T_{\text{compute}} > D$ because no predictions reach the planner in time.

III. SYSTEM ARCHITECTURE

The edge pipeline processes each tick in four sequential stages: fusion, detection, tracking, and prediction. We instantiate each stage with a specific model and measure its per-stage cost scaling on an NVIDIA RTX 4080 SUPER (16 GB).

A. Intermediate Fusion: WorldFusion (Where2Comm)

Each CAV runs a PointPillars [3] encoder locally, producing a BEV feature tensor. After post-backbone compression (NaiveCompressor at $256\times$, reducing the per-vehicle payload from ~ 10 MB to ~ 17 KB), compressed features are uploaded to the edge via 5G Uu. The edge decompresses and fuses using Where2Comm [4] spatial attention, which computes pairwise confidence-weighted attention maps over BEV cells.

Fusion cost scales linearly in K (the number of agents after input filtering): from 3.5 ms at $K = 4$ to 29.5 ms at $K = 32$ for Where2Comm attention. Simple maxout pooling is $20\text{--}40\times$ faster but sacrifices detection quality.

B. Detection

The fused BEV feature is passed through classification and regression heads (shared with the per-agent encoder) to produce 3D bounding boxes via anchor-based decoding and NMS. Detection cost is near-constant in N (~ 1 ms) because it operates on the single fused feature map.

C. Tracking: AB3DMOT

We use AB3DMOT [18] with vectorized Kalman predict/update (replacing the original serialized per-track loop with batched einsum operations), yielding a $1.8\text{--}1.9\times$ speedup. Additional modifications include tightened confirmation gates, duplicate filtering, and kinematic gating.

Tracking cost scales linearly in the number of detected objects M : $c_{\text{track}} \approx 0.7 \text{ ms} \times M$. Through $N = 32$ agents, tracking remains the cheapest stage.

D. Prediction: MTR

We selected Motion Transformer (MTR) [5] over SMART [6] after benchmarking both on OPV2V data. SMART's 143 ms constant-time inference alone exceeds the 100 ms deadline. MTR achieves 4.5 m ADE and 9.4 m FDE at 57 ms. SMART yields 42.7 m ADE and 50.3 m FDE at 137 ms on the same data, reflecting a domain gap (SMART was trained on Waymo, not CARLA-based data).

MTR runs in single-call mode at the edge: one forward pass per tick over all tracked objects, producing 25-step multi-modal trajectories. Compared to the per-CAV invocation used in CMP [1], single-call inference is $1.3 \times$ faster with statistically indistinguishable ADE (4.13 m vs. 4.10 m).

Prediction cost decomposes as $c_{\text{base}} + c_{\text{marginal}} \times |\{i : x_i = 1\}|$, where $c_{\text{base}} \approx 25 \text{ ms}$ (model load, map encoding) and $c_{\text{marginal}} \approx 1.5 \text{ ms}$ per additional object.

E. Cost Model

The total per-tick pipeline cost is:

$$C(K, \mathbf{x}) = c_{\text{filt}}(N) + c_{\text{fuse}}(K) + c_{\text{det}} + c_{\text{track}}(M) + c_{\text{pred}}(\mathbf{x}) \quad (2)$$

where $c_{\text{filt}}(N)$ is the input filter cost ($\sim 0.05 \text{ ms}$ per agent), $c_{\text{fuse}}(K) \approx 1.0 \text{ ms} \times K$, $c_{\text{det}} \approx 1.0 \text{ ms}$, $c_{\text{track}}(M) \approx 0.7 \text{ ms} \times M$, $c_{\text{pred}}(\mathbf{x})$ per above, and $c_{\text{oh}} \approx 2 \text{ ms}$ is scheduling overhead. Coefficients are calibrated from per-stage profiling (§V).

F. Static Pipeline Scaling

?? shows measured per-stage latency vs. N for the static (no adaptation) configuration. The pipeline first exceeds 100 ms at $N = 16$, reaching 184 ms at $N = 64$. Fusion and prediction dominate; tracking remains sub-linear through $N = 64$.

IV. ADAPTIVE CONTROLLER

The controller operates at each tick with two decision variables: $K \in [1, N]$, the number of agents retained after input filtering, and $\mathbf{x} \in \{0, 1\}^M$, a per-object binary indicating fresh MTR inference ($x_i = 1$) or cached/linear prediction ($x_i = 0$). Three mechanisms implement these decisions.

A. Mechanism 1: Input Contribution Filtering

When N incoming agents exceed the budget-feasible count, the controller selects $K < N$ agents whose features contribute most to safety-critical detection. Each agent j is scored by:

$$s_j = (1 - \alpha - \beta) z_j + \alpha u_j + \beta o_j \quad (3)$$

where z_j integrates the agent's detection confidence map over the ego-vehicle conflict zone (a 5-second planned-path corridor), u_j measures the fraction of agent j 's field

of view not covered by other selected agents, and o_j is 1 if agent j is the sole source for an occluded region. We use $\alpha=0.2$, $\beta=0.3$.

The controller searches candidate K values $\{4, 8, 16, 24, 32, N\}$ and selects the K^* that maximizes estimated utility (Equation (1)) given the cost model (Equation (2)). This avoids the pitfall of selecting the largest feasible K , which would over-allocate budget to fusion at the expense of prediction.

B. Mechanism 2: Temporal Amortization

Not every tracked object requires fresh prediction each tick. The controller maintains a prediction cache keyed by track ID. Object i is flagged divergent if:

$$\|\hat{p}_i(t) - p_i^{\text{obs}}(t)\| > \max(d_{\text{floor}}, v_i \cdot \tau) \quad \text{or} \quad |\hat{\theta}_i(t) - \theta_i^{\text{obs}}(t)| > \theta_{\text{thr}} \quad (4)$$

where $\hat{p}_i(t)$ is the cached prediction evaluated at the current tick, $p_i^{\text{obs}}(t)$ is the Kalman-filtered observed position, v_i is the object speed, $\tau = 0.1 \text{ s}$ is the time-domain tolerance, $d_{\text{floor}} = 0.5 \text{ m}$ prevents re-prediction of stationary objects, and $\theta_{\text{thr}} \approx 10$.

The velocity-normalized threshold $v_i \cdot \tau$ allows up to 100 ms of unmodeled velocity drift regardless of speed. A maximum cache age forces refresh even for non-divergent objects.

Objects not flagged divergent reuse their cached trajectory ($q_i = q_{\text{cache}}(a_i)$, decaying with age a_i).

C. Mechanism 3: Risk-Budgeted Scheduling

Among divergent objects, the controller allocates the residual prediction budget to those with highest risk. The continuous risk score for object i is:

$$r_i = v_i \cdot \exp\left(-\frac{d_i}{d_{\text{scale}}}\right) \cdot \omega_i \cdot \max\left(0, 1 - \frac{c_i}{r_{\text{safe}}}\right) \quad (5)$$

where v_i is speed (from the Kalman filter), d_i is distance to the nearest ego vehicle, $d_{\text{scale}} = 20 \text{ m}$ controls proximity decay, $\omega_i \in \{1.0, 2.0\}$ is an occlusion weight (2.0 if the object is visible only to an infrastructure sensor, not to any ego), and c_i is the closest-approach distance between the object's predicted trajectory and any ego's planned path over a 5-second horizon, with safety radius $r_{\text{safe}} = 5 \text{ m}$.

The scheduler ranks divergent objects by r_i descending and greedily assigns $x_i = 1$ until the residual budget $B_{\text{pred}} = D - C_{\text{fixed}}(K)$ is exhausted, where $C_{\text{fixed}}(K)$ is the measured (not estimated) cost of filtering, fusion, detection, and tracking for the current tick. Remaining divergent objects receive linear extrapolation ($q_i = q_{\text{lin}} \approx 0.3$).

D. Tracked-Object Count Estimator

The cost model requires an estimate of M (tracked objects) before fusion and tracking actually run, to evaluate candidate K values. We maintain a rolling window of recent (K, M) observations and compute:

$$\hat{M}(K) = \bar{\rho} \cdot K + \text{slack} \quad (6)$$

Algorithm 1 Adaptive controller (per tick)

Require: N agent features, deadline D , candidate K -set

```
1: if rate fallback active and tick not scheduled then
2:   return cached world model
3: end if
4: Score agents by contribution (Equation (3))
5: for all  $K \in \{4, 8, 16, 24, 32, N\}$  do
6:    $\hat{M} \leftarrow$  estimator (Equation (6))
7:    $\hat{C}_{\text{fix}} \leftarrow c_{\text{filt}}(N) + c_{\text{fuse}}(K) + c_{\text{det}} + c_{\text{track}}(\hat{M})$ 
8:   if  $\hat{C}_{\text{fix}} + c_{\text{oh}} > D$  then skip
9:   end if
10:   $B_{\text{pred}} \leftarrow D - \hat{C}_{\text{fix}} - c_{\text{oh}}$ 
11:  Estimate utility  $U(K, B_{\text{pred}})$ 
12: end for
13:  $K^* \leftarrow \arg \max_K U(K, B_{\text{pred}})$ 
14: if  $K^* = \text{then}$  trigger rate fallback
15: end if
16: Run fusion on top- $K^*$  agents
17: Run detection and tracking
18: Compute divergence gates (Equation (4))
19:  $B_{\text{res}} \leftarrow D - C_{\text{measured}} \triangleright$  actual residual
20: Rank divergent objects by risk  $r_i$  (Equation (5))
21: Greedily assign  $x_i=1$  within  $B_{\text{res}}$ 
22: Run MTR on selected objects; linear fallback for rest
23: return updated world model
```

where $\bar{\rho}$ is the mean tracks-per-agent ratio over the last 20 ticks and slack = $\max(0.1 \cdot \bar{\rho} \cdot K, 2)$ provides headroom for workload spikes. Cold start defaults to 4 tracks per agent.

E. Rate Adaptation (Fallback)

If no candidate K yields a feasible allocation (i.e., even $K = 4$ would exceed the deadline), the controller reduces the global processing rate: it skips $k - 1$ out of every k ticks, where $k \in \{2, 4, 5, 10\}$ is chosen from the p95 of recent compute times. Rate adaptation is a last resort that trades temporal resolution for deadline compliance.

F. Algorithm Summary

G. What the Controller Does Not Claim

The controller is a two-level greedy heuristic, not globally optimal. It does not provide worst-case deadline guarantees (empirical compliance is reported). Safety priority is best-effort: high-risk objects receive prediction budget first, but the risk score is a calibrated surrogate, not a formal safety certificate. The greedy gap relative to an ILP oracle is reported in §V for small instances where the oracle is tractable.

V. EVALUATION

A. Experimental Setup

Hardware. All measurements are on an NVIDIA RTX 4080 SUPER (16GB VRAM) with an AMD Ryzen 9 7950X CPU.

Pipeline. WorldFusion (Where2Comm attention fusion), AB3DMOT tracking (vectorized Kalman), MTR prediction (25-step multi-modal). Post-backbone features are real, extracted from the OPV2V [7] validation split through the trained WorldFusion encoder (post-backbone, 256-channel, shrink-convolved).

Agent scaling. $N \in \{4, 8, 16, 24, 32, 48, 64\}$. For N exceeding the number of unique OPV2V CAVs, features are replicated with distinct transformations to preserve workload fidelity.

Ablation configurations. Five policies isolate each mechanism: (i) *static*: full pipeline, no adaptation; (ii) *filter_only*: input contribution filter with $K_{\text{max}}=16$, no prediction adaptation; (iii) *amort_only*: divergence gating only, no filter, no risk budget; (iv) *risk_only*: risk-budgeted scheduling only (30 ms prediction budget), no filter, no amortization; (v) *adaptive_all*: all three mechanisms enabled ($K_{\text{max}}=16$, amortization, 30 ms prediction budget).

Ticks. 30 ticks per configuration, with steady-state metrics computed over the last 8 ticks (after tracker warmup).

B. Per-Stage Latency Scaling

?? reports per-stage latency for the static pipeline. Fusion scales linearly with N (Where2Comm pairwise attention), tracking scales linearly with the number of detected objects M , and MTR prediction scales with the number of tracked objects requiring inference. Detection is near-constant (~ 1 ms) because it operates on the fused feature map.

The pipeline first exceeds 100 ms at $N=16$ (149 ms) and reaches 302 ms at $N=32$. Beyond $N=32$, timing is estimated from the per-stage cost model (Equation (2)) calibrated on the measured range.

C. Five-Way Ablation

???? present end-to-end latency and deadline compliance for each configuration.

Static baseline. Fails at $N \geq 16$; latency grows to 184 ms at $N=64$. The non-monotonicity at $N=32$ (104 ms) reflects variation in detection count across ticks.

Filter only. Capping agents at $K=16$ restores compliance for $N \geq 24$ but does not help at $N=16$ (where $K=N$). The filter itself adds < 2 ms even at $N=64$.

Amortization only. Reduces prediction cost by reusing cached trajectories for non-divergent tracks. Effective through $N=32$ (100% compliance) but loses compliance at $N=48$ because fusion cost alone approaches the deadline.

Risk-budget only. Caps prediction at 30 ms by serving only the highest-risk divergent tracks. Effective through $N=32$ but fusion cost exceeds the budget at $N=48$.

TABLE I: Predictor comparison on OPV2V data.

Model	ADE (m)	FDE (m)	Latency (ms)
SMART (Waymo-trained)	42.70	50.25	137
MTR (multi-call, CMP)	4.10	9.00	74
MTR (single-call, edge)	4.13	8.85	57

Adaptive (all three). The only configuration achieving 100% deadline compliance at every N . At $N=64$, end-to-end latency is 50 ms (50% headroom), because the filter caps fusion cost while amortization and risk budgeting cap prediction cost.

D. Quality Metrics: Fixed-GT Evaluation

To avoid the pitfalls of nearest-neighbor GT matching (ghost predictions matched to arbitrary objects, missing predictions silently ignored), we use a fixed-GT evaluation:

- For each GT object, find the best matching prediction within a 2 m radius.
- Unmatched GT objects contribute a 10 m miss penalty to ADE.
- The miss rate (fraction of GT objects without a matching prediction) is reported explicitly.
- Quality is stratified by risk level: objects within 30 m of the ego origin are high-risk; others are low-risk.

?? reports steady-state quality metrics. **Insert quality numbers from Multi-V2X evaluation.**

E. Predictor Selection: MTR over SMART

Table I compares MTR and SMART on the same OPV2V evaluation set. SMART’s 137 ms inference time alone exceeds the 100 ms deadline. Its 42.7 m ADE (vs. MTR’s 4.5 m) reflects a Waymo-to-CARLA domain gap. MTR’s single-call edge-mode inference (56.8 ms, 4.13 m ADE) is statistically equivalent to the multi-call CMP-style invocation (73.8 ms, 4.10 m ADE) at $1.3\times$ lower latency.

F. V2V Networking Validation

We run ns-3 5G-LENA NR V2X Mode 2 sidelink simulations to validate the analytical argument in §II. At 200 KB intermediate fusion payloads, PRR = 0 at all N : the payload exceeds the sidelink transport block size. At 17 KB compressed payloads, PRR degrades from 0.30 ($N=2$) to 0.006 ($N=64$). These results confirm that V2V intermediate fusion is unviable at dense scale, independent of the MAC protocol or bandwidth allocation.

G. Cost Model Validation

Insert predicted-vs-measured latency scatter for each stage (c_{fuse} , c_{track} , c_{pred}). Target: $R^2 > 0.95$ on held-out ticks.

H. Greedy Gap

For $N \leq 8$ where ILP is tractable, report gap between greedy utility and oracle optimal. Preliminary: gap < 5% in all tested instances.

VI. DISCUSSION AND LIMITATIONS

A. Mechanism Interaction

The three mechanisms are complementary, not redundant. Input filtering bounds fusion cost, which is the first stage to exceed the deadline as N grows. Temporal amortization reduces prediction cost at any N by exploiting inter-tick coherence. Risk budgeting allocates the remaining prediction budget to the objects that matter most for planner safety. The ablation (??) shows that no single mechanism achieves 100% compliance across the full N range; the combination does.

B. Generalization

The cost model coefficients (Equation (2)) are hardware-specific (calibrated on RTX 4080 SUPER). Deploying on different MEC hardware (e.g., NVIDIA Orin, T4) requires re-profiling per-stage latencies. The controller structure and decision variables are hardware-agnostic: the same K -search and risk-budgeted scheduling apply once the cost model is re-calibrated.

The input contribution filter currently uses a BEV conflict-zone integral. Alternative scoring functions (e.g., information-theoretic measures, learned attention) could replace it without changing the controller’s outer loop.

C. Limitations

Offline evaluation only. All measurements are from an offline profiler replaying recorded features. Closed-loop CARLA evaluation (where planner decisions feed back into vehicle behavior) is in progress.

Synthetic detection workload. Detections fed to the tracker are synthetically generated from ground-truth trajectories, not from the fusion model’s actual output. The timing measurements use real features and real model inference, but the detection-to-tracking interface is simulated.

Divergence thresholds are heuristic. The velocity-normalized threshold $v_i \cdot \tau$ (Equation (4)) is manually tuned. A learned threshold policy conditioned on object type and scene context may improve amortization rates without sacrificing quality.

Risk score not validated against closed-loop outcomes. The multiplicative risk score (Equation (5)) is a surrogate calibrated against offline prediction error, not against planner collision rates. A learned risk function trained on closed-loop data would strengthen the safety argument.

Cache coherence. Tracker ID switches invalidate the prediction cache. The current implementation treats an ID switch as a new object (cold start), which temporarily increases prediction cost. Integrating re-identification could mitigate the penalty.

Dataset scale. OPV2V provides at most 2–5 real CAVs per scene. For $N > 5$, we replicate features to synthesize larger agent counts. Evaluation on Multi-V2X [8] (up to 31 real CAVs per frame, 410 total CAVs, 56 RSUs) is in progress and will provide genuine dense-scale workloads.

D. Comparison with Prior Work

CMP [1] is the closest system: it combines CoBEVT fusion, AB3DMOT tracking, and MTR prediction. CMP reports 67.3 ms total latency on an A6000 at 2–7 agents, with no deadline constraint and no adaptation mechanism. Our work extends the evaluation to $N=64$, introduces the deadline cliff characterization, and proposes the adaptive controller that maintains compliance at dense scale.

EMP [9] and Harbor [10] address edge-hosted cooperative perception but stop at detection (no tracking or prediction), report latency at small N , and do not model the joint-pipeline deadline cliff.

VII. RELATED WORK

A. Cooperative Perception

Cooperative perception architectures trade off bandwidth against perceptual fidelity. Early fusion (raw data exchange) is bandwidth-prohibitive. Late fusion (object-level sharing) discards spatial context needed for occluded-object detection. Intermediate fusion dominates recent work: Where2Comm [4] uses spatial-confidence attention maps, V2X-ViT [11] applies vision transformers to BEV features, and CoBEVT [12] fuses multi-scale BEV representations with axial attention.

CMP [1], [2] is the only prior system that unifies cooperative perception, multi-object tracking (AB3DMOT), and trajectory prediction (MTR) into a single evaluated pipeline. CMP reports results at 2–7 agents on OPV2V and V2V4Real, with no deadline constraint. Our work extends the same pipeline family to 4–64 agents, characterizes the resulting deadline cliff, and introduces adaptation mechanisms that maintain feasibility at dense scale.

B. Edge-Hosted Perception Systems

EMP [9] demonstrated edge-hosted LiDAR fusion at 93 ms end-to-end on a real intersection, but evaluates detection only (no tracking or prediction). Harbor [10] tests a hybrid V2I/V2V architecture on a 3-vehicle Mcity deployment, using frame timeouts rather than a compute-budget controller. F-Cooper [13] evaluates point-cloud feature sharing at the edge but does not address the computational scaling problem. EdgeCooper [14] considers communication-aware scheduling for edge cooperative perception but evaluates at $N \leq 5$.

None of these systems evaluate the full pipeline (fusion + tracking + prediction) at $N > 10$, nor do they model the joint-pipeline deadline cliff that emerges from cross-stage compute coupling.

C. Adaptive Compute for Autonomous Vehicles

Single-vehicle adaptive compute has been studied in the context of streaming perception [15] and modular self-driving platforms [16]. These approaches adjust model complexity or scheduling policy per frame based on deadline proximity. They operate within a single vehicle’s compute budget and do not address the multi-agent cooperative setting where N agents share one edge deadline.

D. Multi-Object Tracking

Classical multi-object tracking uses Kalman filtering with Hungarian association (SORT [17], AB3DMOT [18]). Recent work explores learned association and state-space models for temporal dynamics [19], [20]. We use AB3DMOT with vectorized Kalman operations ($1.8\times$ speedup over the original per-track loop), which provides sufficient tracking quality while remaining the cheapest pipeline stage.

E. Trajectory Prediction

MTR [5] and MTR++ [21] use transformer decoders with motion-query attention to produce multi-modal trajectory forecasts. SMART [6] tokenizes trajectories for autoregressive generation. We evaluated both on OPV2V data: MTR achieves 4.5 m ADE at 57 ms, while SMART yields 42.7 m ADE at 137 ms (reflecting a Waymo-to-CARLA domain gap and exceeding the 100 ms deadline on its own). MTR is the only viable option for deadline-constrained edge inference in our pipeline.

VIII. CONCLUSION

We characterize the deadline cliff that emerges when a cooperative world-model pipeline (WorldFusion attention fusion, AB3DMOT tracking, MTR transformer prediction) is deployed on an edge server at dense scale. On commodity hardware, the static pipeline exceeds the 100 ms planner deadline at $N \approx 10$ agents; at $N=64$ it reaches 184 ms.

Our adaptive controller selects the number of contributing agents (input filter K) and the set of objects receiving fresh prediction (risk-budgeted $\mathbf{x}$) to maximize risk-weighted utility within the deadline budget. Three mechanisms interact: input contribution filtering bounds fusion cost, temporal amortization reuses cached predictions for stable tracks, and risk-budgeted scheduling prioritizes the objects most relevant to planner safety.

In a five-way offline ablation across $N \in \{4, \dots, 64\}$, the combined controller is the only configuration achieving 100% deadline compliance at every scale, while maintaining prediction quality for high-risk objects. Ongoing work extends this evaluation to Multi-V2X (up to 31 real CAVs per frame) and closed-loop CARLA simulation to validate the safety argument end-to-end.

REFERENCES

- [1] Z. Wu, Y. Wang, H. Ma, Z. Li, H. Qi, and J. Li, "Cmp: Cooperative motion prediction with multi-agent communication," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1–8, 2025.
- [2] Z. Wang, Y. Wang, Z. Wu, H. Ma, Z. Li, H. Qiu, and J. Li, "Cmp: Cooperative motion prediction with multi-agent communication," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3876–3883, 2025.
- [3] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *CVPR*, 2019.
- [4] Y. Hu, S. Fang, Z. Lei, Y. Zhong, and S. Chen, "Where2comm: Communication-efficient collaborative perception via spatial confidence maps," in *NeurIPS*, 2022.
- [5] S. Shi, L. Jiang, D. Dai, and B. Schiele, "Motion transformer with global intention localization and local movement refinement," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] W. Wu, X. Feng, Z. Gao, and Y. Kan, "Smart: Scalable multi-agent real-time simulation via next-token prediction," in *NeurIPS*, 2024.
- [7] R. Xu, H. Xiang, X. Xia, X. Han, J. Li, and J. Ma, "Opv2v: An open benchmark dataset and fusion pipeline for perception with vehicle-to-vehicle communication," in *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [8] R. Li and X. Pei, "Multi-v2x: A large scale multi-modal multi-penetration-rate dataset for cooperative perception," in *IEEE Intelligent Vehicles Symposium (IV)*, 2025, to appear.
- [9] X. Zhang, A. Zhang, J. Sun, X. Zhu, Y. E. Guo, F. Qian, and Z. M. Mao, "Emp: edge-assisted multi-vehicle perception," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, ser. MobiCom '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 545–558. [Online]. Available: <https://doi.org/10.1145/3447993.3483242>
- [10] R. Zhu, X. Zhu, A. Zhang, X. Zhang, J. Sun, F. Qian, H. Qiu, Z. M. Mao, and M. Lee, "Boosting collaborative vehicular perception on the edge with vehicle-to-vehicle communication," in *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*, ser. SenSys '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 141–154. [Online]. Available: <https://doi.org/10.1145/3666025.3699328>
- [11] R. Xu, H. Xiang, Z. Tu, X. Xia, M.-H. Yang, and J. Ma, "V2x-vit: Vehicle-to-everything cooperative perception with vision transformer," in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXXIX*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 107–124. [Online]. Available: https://doi.org/10.1007/978-3-031-19842-7_7
- [12] R. Xu, Z. Tu, H. Xiang, W. Shao, B. Zhou, and J. Ma, "Cobevt: Cooperative bird's eye view semantic segmentation with sparse transformers," in *CoRL*, 2022.
- [13] Q. Chen, X. Ma, S. Tang, J. Guo, Q. Yang, and S. Fu, "F-cooper: feature based cooperative perception for autonomous vehicle edge computing system using 3d point clouds," in *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*, ser. SEC '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 88–100. [Online]. Available: <https://doi.org/10.1145/3318216.3363300>
- [14] G. Luo, C. Shao, N. Cheng, H. Zhou, H. Zhang, Q. Yuan, and J. Li, "Edgcooper: Network-aware cooperative lidar perception for enhanced vehicular awareness," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 1, pp. 207–222, 2024.
- [15] M. Li, Y.-X. Wang, and D. Ramanan, "Towards streaming perception," in *ECCV*, 2020.
- [16] I. Gog, S. Kalra, P. Schafhalter, M. A. Wright, J. E. Gonzalez, and I. Stoica, "Pylot: A modular platform for exploring latency-accuracy tradeoffs in autonomous vehicles," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [17] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [18] X. Weng, J. Wang, D. Held, and K. Kitani, "3d multi-object tracking: A baseline and new evaluation metrics," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 10 359–10 366.
- [19] H.-W. Huang, C.-Y. Yang, W. Chai, Z. Jiang, and J.-N. Hwang, "Mambatrack: A simple baseline for multiple object tracking with state space model," in *AAAI*, 2025.
- [20] K. Xie, Z. Huang, Z. Zhou, and J. Ma, "Trackssm: A general motion predictor by state-space model," in *AAAI*, 2025.
- [21] S. Shi, L. Jiang, D. Dai, and B. Schiele, "Mtr++: Multi-agent motion prediction with symmetric scene modeling and paired data augmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, 2024.